

Politechnika Łódzka



IAPŁ

INSTYTUT AUTOMATYKI
POLITECHNIKI ŁÓDZKIEJ

Zakład sterowania robotów

ROBOTY MOBILNE

Instrukcja laboratoryjna 4

Uruchomienie i obsługa środowiska



23 października 2025

Spis treści

1	Wprowadzenie	2
1.1	Przydatne skróty klawiaturowe	2
1.2	Współdzielenie zasobów pomiędzy komputerem natywnym a środowiskiem w kontenerze	2
1.3	Podstawy pracy z terminalem	4
2	Uruchomienie kontenera	5
2.1	Wybór systemu operacyjnego.	5
2.2	Sprawdzenie połączeń sieciowych	6
2.3	Otwieranie terminala we właściwej lokalizacji	7
2.4	Uruchomienie kontenera	8
2.5	Połączenie przez przeglądarkę	9
2.6	Praca w kontenerze	9
3	Konfiguracja sieciowa środowiska laboratoryjnego	10
3.1	ROS_DOMAIN_ID – separacja komunikacji	11
3.2	Konfiguracja sieciowa robotów i stanowisk	11
3.3	Konfiguracja pliku <i>.bashrc</i> w kontenerze	12
4	Uruchomienie środowiska Jupyter Notebook w Dockerze	14
4.1	Uruchamianie Jupyter Notebook	14
4.2	Problemy przy uruchamianiu	14
4.3	Struktura katalogów w Jupyter Notebook	14
4.3.1	Uruchamianie ćwiczeń	15
5	Obsługa Jupyter Notebook	15
5.1	Zapisywanie zmian	15
5.2	Ikony i przyciski w pasku narzędzi	16
5.3	Typy komórek	16
5.4	Numeracja komórek	17
5.5	Komórki z <i>In [*]</i> – ważne ostrzeżenie	17
6	Uruchomienie robota	18
6.1	Przygotowanie robota	18
6.2	Połączenie z robotem	19
6.3	Konfiguracja środowiska na robocie	19
7	Sterowanie robotem	20
7.1	Interfejs graficzny w Jupyter Notebook	20
7.2	Sterowanie z terminala	21
7.3	Interfejs graficzny <i>rqt_publisher</i>	21

8	Weryfikacja działania kinematyki w ROS 2	22
8.1	Weryfikacja kinematyki odwrotnej	22
8.2	Weryfikacja kinematyki prostej	23

1 Wprowadzenie

1.1 Przydatne skróty klawiaturowe

Podczas pracy w laboratorium często korzystamy z terminala, Jupyter Notebooka czy edytorów tekstu. Aby przyspieszyć i ułatwić sobie pracę, warto znać kilka podstawowych skrótów klawiaturowych. Dzięki nim można szybciej uruchamiać terminal, kopiować, wklejać lub komentować fragmenty kodu.

Tab. 1.1: Przydatne skróty klawiaturowe

Skrót	Gdzie działa	Znaczenie / zastosowanie	Uwagi praktyczne
Ctrl+Alt+T	System (Linux) - tylko komputer główny a nie docker	Otwiera nowy terminal	Najszybszy sposób uruchomienia konsoli, nie trzeba szukać w menu.
Ctrl+Shift+C	Terminal	Kopiuje zaznaczony tekst w terminalu	Uwaga: zwykle Ctrl+C w terminalu nie kopiuje, tylko przerywa działanie programu.
Ctrl+Shift+V	Terminal	Wkleja tekst do terminala	Przydatne do wklejania dłuższych komend; nie działa zwykle Ctrl+V.
Ctrl+C	Instrukcja, Jupyter, pliki tekstowe	Kopiowanie zaznaczonego tekstu	Standardowy skrót znany z większości aplikacji.
Ctrl+V	Jupyter, edytory tekstu	Wklejanie skopiowanego tekstu	Działa w edytorach i przeglądarce, ale nie w terminalu (tam używamy Ctrl+Shift+V).
Ctrl+/ Ctrl+Enter	Jupyter Notebook	Komentowanie lub odkomentowywanie zaznaczonego kodu	Bardzo przydatne podczas testowania – można szybko wyłączyć fragment kodu.
Ctrl+Enter	Jupyter Notebook	Uruchomienie zaznaczonej komórki	Wynik pojawia się pod komórką; kursor zostaje w tej samej komórce.
Shift+Enter	Jupyter Notebook	Uruchomienie zaznaczonej komórki i przejście do kolejnej	Najczęściej używany skrót – przyspiesza pracę z notatnikiem.

1.2 Współdzielenie zasobów pomiędzy komputerem natywnym a środowiskiem w kontenerze

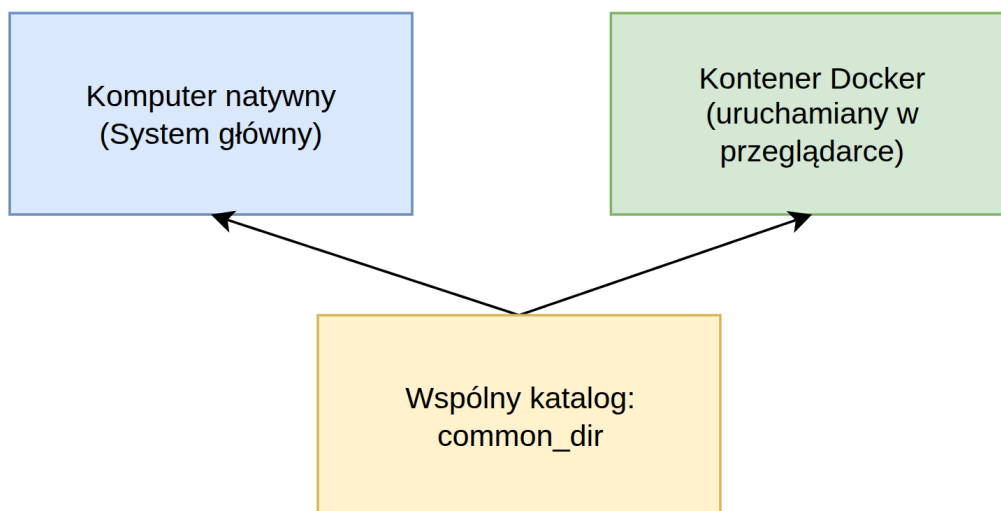
Podczas zajęć laboratoryjnych korzystamy ze specjalnie przygotowanego środowiska uruchamianego w kontenerze **Dockera**, dostępnego w przeglądarce. Aby ułatwić pracę i zapewnić możliwość wymiany plików pomiędzy głównym systemem a kontenerem, przygotowano wspólny katalog.

Wspólny katalog

Na **Pulpicie** komputera w systemie natywnym znajduje się katalog:

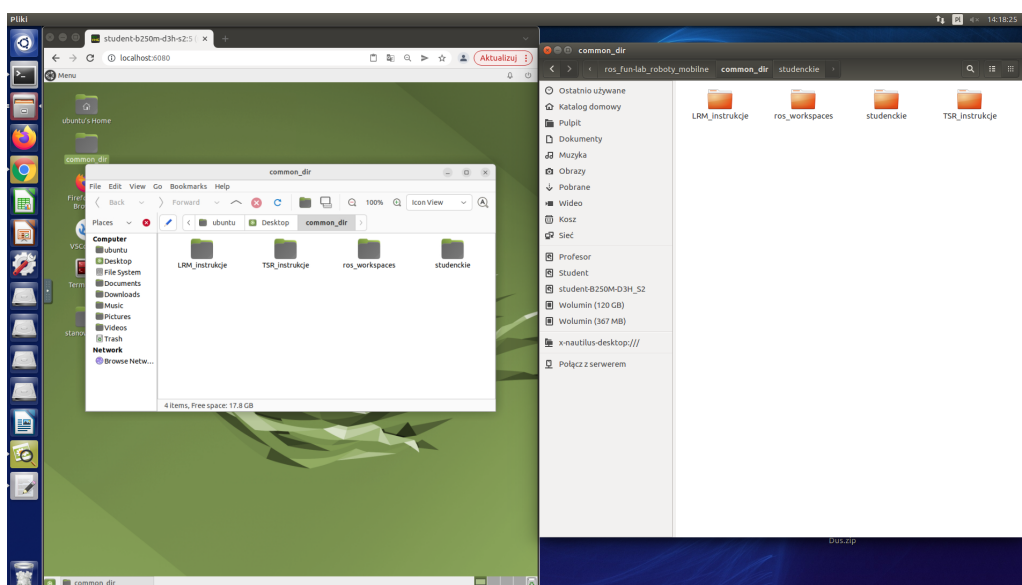
```
1 materials_do_zajec/ros-fun-lab_roboty_mobilne/common_dir
```

Kod źródłowy 1: Lokalizacja współdzielonego katalogu

Rys. 1.1: Współdzielony katalog *common_dir*

Katalog *common_dir* pełni rolę współdzielonego katalogu – wszystkie pliki, które zostaną do niego zapisane, są automatycznie widoczne zarówno:

- w systemie głównym (po prawej stronie na Rys. 1.2),
- jak i w środowisku kontenera uruchomionego w przeglądarce (po lewej stronie na Rys. 1.2).

Rys. 1.2: Współdzielony katalog *common_dir*

Oznacza to, że jest to jedyne miejsce, w którym należy zapisywać pliki, jeśli chcemy mieć do nich dostęp w obu środowiskach.

Organizacja pracy studentów

Przed rozpoczęciem zajęć każdy student powinien:

1. Utworzyć własny katalog w folderze **studenckie**, znajdującym się wewnątrz katalogu *common_dir*. Dzięki temu materiały będą uporządkowane i każdy student będzie pracował we własnym katalogu.
2. Pobrać dostarczone materiały z platformy **Wikamp** lub skopiować katalog *LRM_instrukcje* widoczny w folderze współdzielonym. Kopiowanie katalogu zapewnia możliwość pracy na własnej wersji plików.

3. Należy pracować wyłącznie na swojej kopii katalogu z instrukcjami.

- Katalog *LRM_instrukcje* pełni rolę szablonu.
- Nie wolno go edytować ani zapisywać w nim własnych plików, ponieważ w przypadku stwierdzenia jego modyfikacji zostanie przywrócony do wartości domyślnych, a wszystkie zmiany zostaną utracone.

1.3 Podstawy pracy z terminalem

Terminal w systemie Ubuntu jest narzędziem, które umożliwia bezpośrednią komunikację z systemem operacyjnym poprzez **wiersz poleceń**. Dzięki niemu użytkownik może:

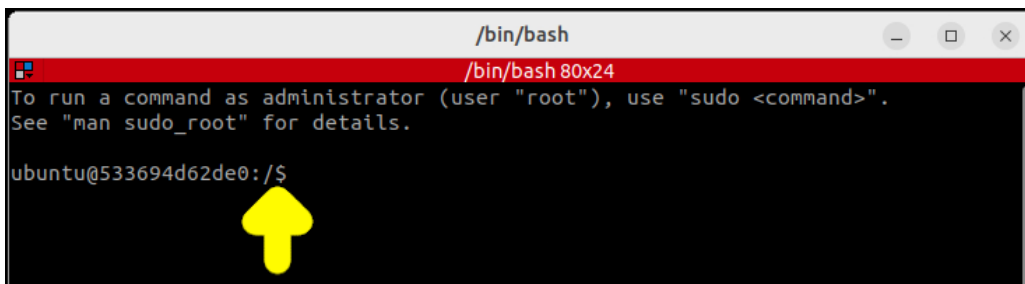
- uruchamiać programy,
- wykonywać polecenia administracyjne,
- pisać i uruchamiać skrypty,
- konfigurować środowisko pracy.

Podstawowym elementem terminala jest znak zachęty (*ang. prompt*). To symbol lub tekst, który informuje użytkownika, że terminal jest gotowy do przyjęcia polecenia. W terminalu pojawia się znak zachęty (np. `$` albo `ubuntu@533694d62de0:/$`) i migający kursor oczekujący na wykonanie wpisu.

Terminal może działać w dwóch trybach:

1. Tryb wprowadzania poleceń

- widoczny jest znak zachęty,
- użytkownik może pisać komendy i zatwierdzać je klawiszem **Enter**,
- każda komenda zostaje wykonana przez system, a wynik jest wyświetlany w tym samym oknie.



Rys. 1.3: Terminal - tryb wprowadzania poleceń.

2. Tryb pracy programu

- terminal nie wyświetla znaku zachęty,
- ekran pokazuje tylko wynik działania uruchomionego programu (np. logi systemowe, aplikacja pełnoekranowa, interaktywne narzędzie),
- klawiatura nie służy do wpisywania poleceń, ale do sterowania aktualnie działającym procesem.

```

/bin/bash
/bin/bash 84x24
azania_kinematyki_v2/stanowisko_1
[I 16:09:48.660 NotebookApp] Kernel started: a1c5f220-40d6-4a22-b786-e3a6fb78c469, n
ame: python3
[I 16:11:48.658 NotebookApp] Saving file at /Desktop/common_dir/moje_rozwiazania_kin
ematyki_v2/stanowisko_1/Untitled.ipynb
[I 16:21:58.852 NotebookApp] Kernel interrupted: a1c5f220-40d6-4a22-b786-e3a6fb78c46
9
[I 16:22:13.650 NotebookApp] Starting buffering for a1c5f220-40d6-4a22-b786-e3a6fb78
c469:218139a886224452810bf53594382671
[I 16:22:13.877 NotebookApp] Kernel restarted: a1c5f220-40d6-4a22-b786-e3a6fb78c469
[I 16:22:13.884 NotebookApp] Restoring connection for a1c5f220-40d6-4a22-b786-e3a6fb
78c469:218139a886224452810bf53594382671
[I 16:22:14.393 NotebookApp] Replaying 3 buffered messages
[I 16:23:48.659 NotebookApp] Saving file at /Desktop/common_dir/moje_rozwiazania_kin
ematyki_v2/stanowisko_1/Untitled.ipynb
[I 16:25:48.653 NotebookApp] Saving file at /Desktop/common_dir/moje_rozwiazania_kin
ematyki_v2/stanowisko_1/Untitled.ipynb
[I 16:27:13.405 NotebookApp] Kernel interrupted: a1c5f220-40d6-4a22-b786-e3a6fb78c46
9
[I 16:27:48.658 NotebookApp] Saving file at /Desktop/common_dir/moje_rozwiazania_kin
ematyki_v2/stanowisko_1/Untitled.ipynb
[I 16:33:48.659 NotebookApp] Saving file at /Desktop/common_dir/moje_rozwiazania_kin
ematyki_v2/stanowisko_1/Untitled.ipynb

```

Rys. 1.4: Terminal - tryb pracy programu.

Aby powrócić do wpisywania komend należy uruchomić nowy terminal lub zakończyć działanie bieżącego programu (np. skrótem **Ctrl+C**).

2 Uruchomienie kontenera

Poniżej opisano krok po kroku, jak poprawnie uruchomić środowisko laboratoryjne działające w kontenerze Dockera.

2.1 Wybór systemu operacyjnego.

Po włączeniu komputera pojawia się ekran wyboru systemu: Ubuntu lub Windows.

- Domyślnie ustawiony powinien być system **Ubuntu**.
- Jeśli zaznaczony jest inny system, należy **strzałkami góra/dół** wybrać Ubuntu i wcisnąć **Enter**.
- Jeżeli użytkownik nic nie zrobi, po kilku sekundach automatycznie uruchomi się aktualnie podświetlony system.

Następnie logujemy się na konto użytkownika:

- Nazwa użytkownika: **student**
- Hasło: **student**



Rys. 2.1: Widok pulpitu po zalogowaniu

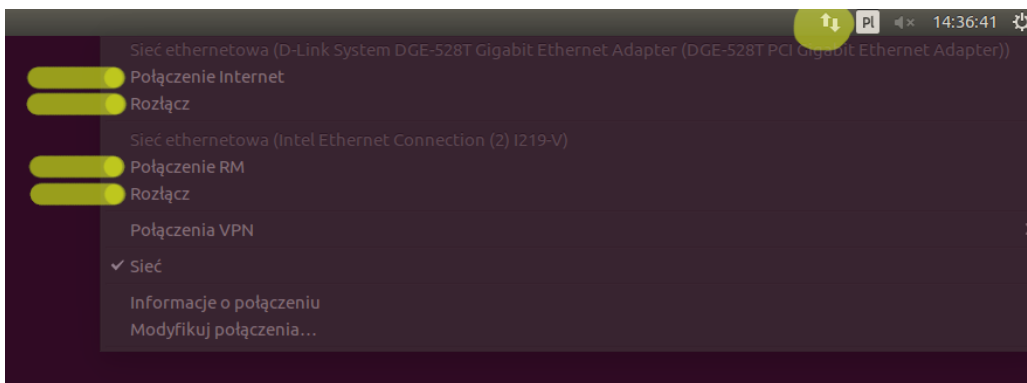
2.2 Sprawdzenie połączeń sieciowych

Aby kontener działał prawidłowo, muszą być aktywne dwa połączenia sieciowe:

- **Internet** – odpowiada za wyświetlanie Dockera w przeglądarce, Samo połączenie bez RM umożliwia dostęp do internetu.
- **RM** – zapewnia komunikację z robotami.

Jak to sprawdzić?

1. W prawym górnym rogu ekranu klikamy ikonę połączeń sieciowych (strzałka w górę i w dół).



Rys. 2.2: Rozwinięcie informacji o aktywnych połączeniach sieciowych.

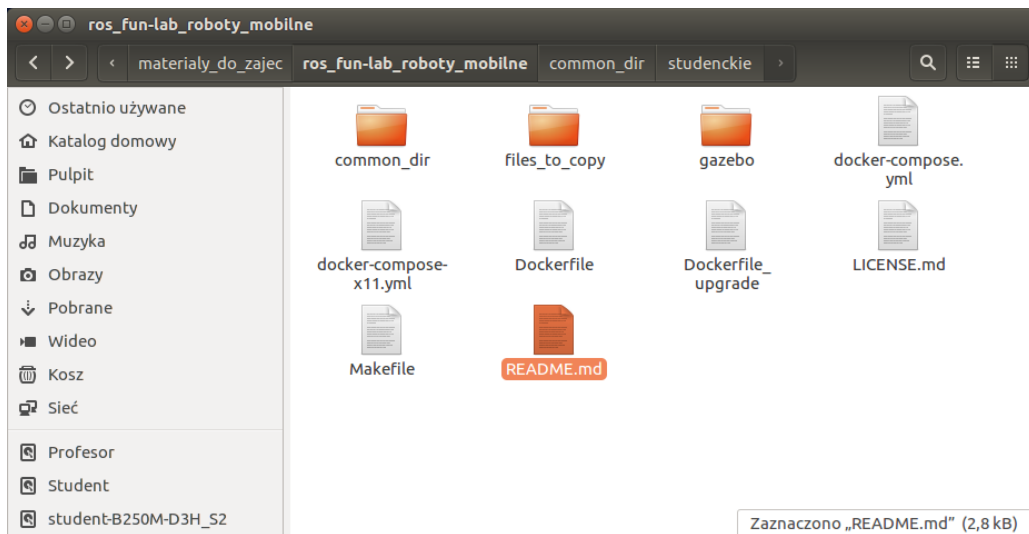
2. Rozwija się menu z listą dostępnych sieci.
3. Rozłączyć obie sieci i podłączyć w następującej kolejności „**Połączenie RM**” a później „**Połączenie Internet**”. Połączenie sieci w innej kolejności może powodować problem z uruchomieniem dockera przy ciskiem **Połącz** w przeglądarce.
4. Pod każdą z sieci powinien być widoczny napis „**Rozłącz**” – oznacza to, że **sieci są włączone**. Jest to prawidłowa konfiguracja.

2.3 Otwieranie terminala we właściwej lokalizacji

Aby uruchomić środowisko, musimy otworzyć terminal w odpowiednim katalogu. Można to zrobić na dwa sposoby:

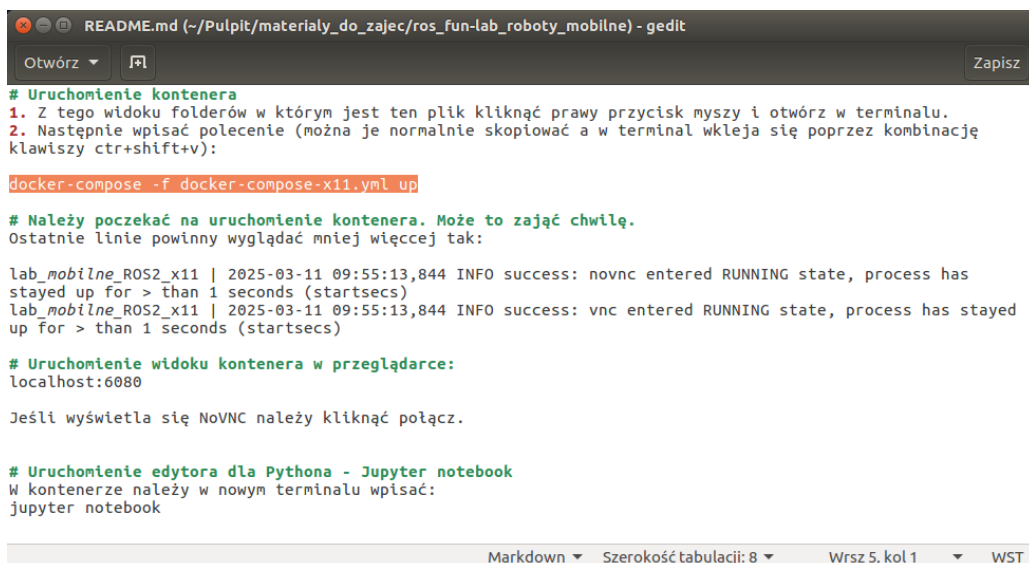
I sposób

1. Na pulpicie znaleźć katalog: *materiały_do_zajec*.
2. Otworzyć kolejne katalogi: *ros_fun-lab_roboty_mobilne* → *common_dir*.
3. W katalogu znajduje się plik *README.md* – są w nim informacje, jak uruchomić środowisko i jaki adres wpisać w przeglądarce.



Rys. 2.3: Współdzielony katalog *common_dir* z zaznaczonym plikiem *README.md*

4. Po otwarciu pliku znajduje się w nim informacja o poleceniu jakie należy wpisać w terminalu, aby uruchomić dockera.



Rys. 2.4: Otwarty plik *README.md*

5. Klikamy prawym przyciskiem myszy w pustym miejscu katalogu i wybieramy opcję „Otwórz w terminalu”.

6. Terminal uruchomi się od razu we właściwej lokalizacji.



Rys. 2.5: Prawidłowa ścieżka w terminalu przed uruchomieniem kontenera.

II sposób

1. Uruchomić nowy terminal skrótem **Ctrl+Alt+T**.
2. Wpisać komendę i zatwierdzić **Enter**:

```
1 cd /home/student/Pulpit/materialy_do_zajec/ros_fun-lab_roboty_mobilne/
```

Kod źródłowy 2: Ustawienie właściwej ścieżki do katalogu dockera z poziomu terminala

3. Po przejściu do katalogu terminal będzie wskazywał tę samą ścieżkę co w metodzie pierwszej.

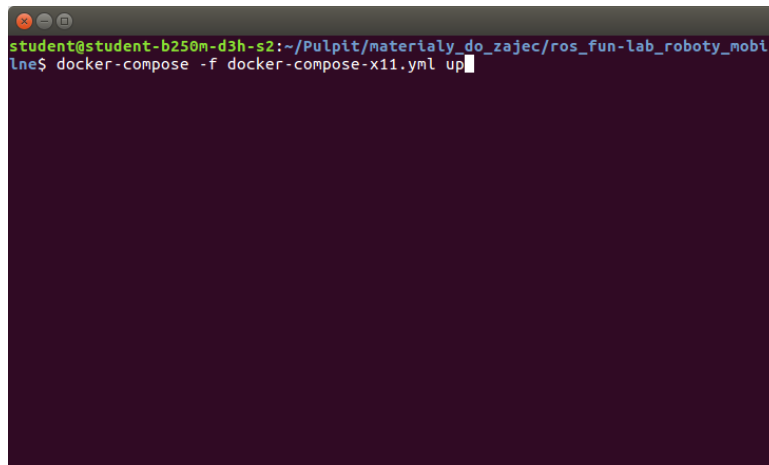
2.4 Uruchomienie kontenera

W terminalu wpisujemy polecenie:

```
1 docker-compose -f docker-compose-x11.yml up
```

Kod źródłowy 3: Polecenie do uruchomienia kontenera.

- Po zatwierdzeniu **Enter** zacznie się uruchamianie środowiska.
- W terminalu pojawią się logi startowe kontenera.



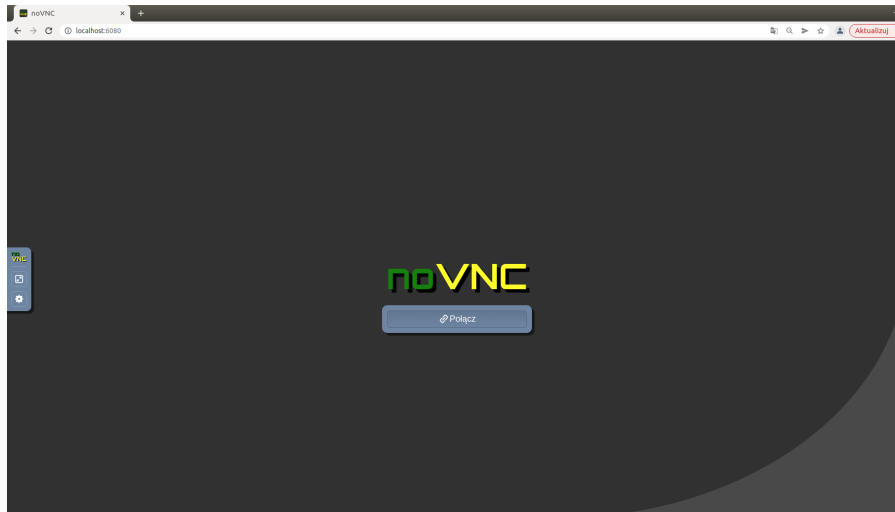
Rys. 2.6: Wpisanie polecenia w terminalu uruchamiającego kontener.

2.5 Połączenie przez przeglądarkę

1. Uruchamiamy przeglądarkę (np. Google Chrome, Firefox).
2. W pasku adresu wpisujemy:

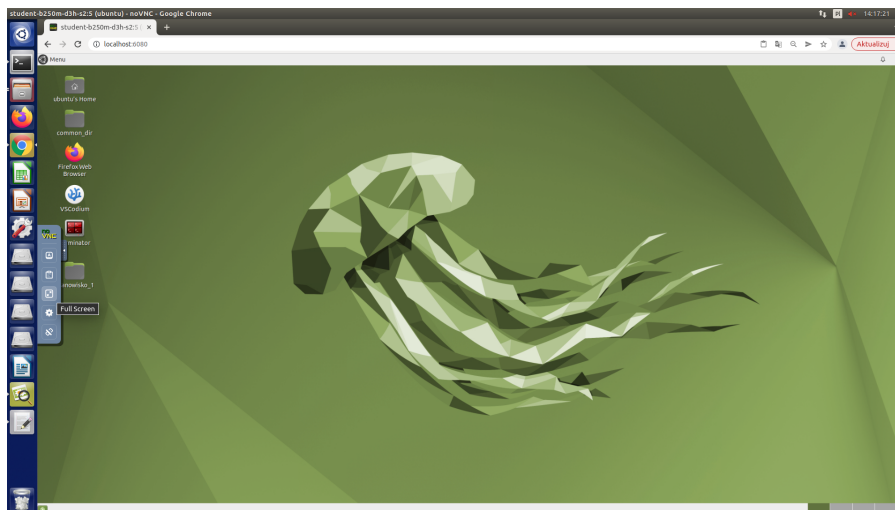
```
1 localhost:6080
```

Kod źródłowy 4: Ustawienie adresu w przeglądarce.



Rys. 2.7: Widok w przeglądarce po wpisaniu adresu.

3. Klikamy przycisk **Połącz**.



Rys. 2.8: Widok w przeglądarce po nawiązaniu połączenia.

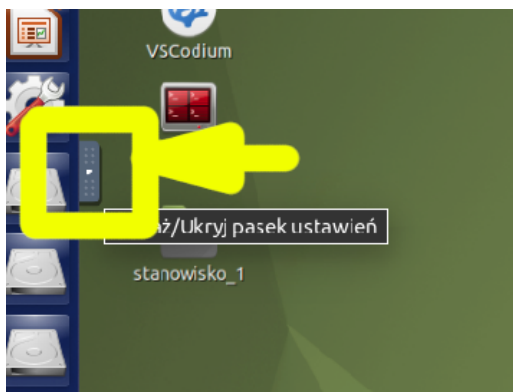
Kontener został poprawnie uruchomiony – od tej chwili pracujemy już tylko w środowisku kontenera.

2.6 Praca w kontenerze

— Skrót **Ctrl+Alt+T** w kontenerze **nie działa** – otwiera terminal systemu głównego, a nie Dockera.

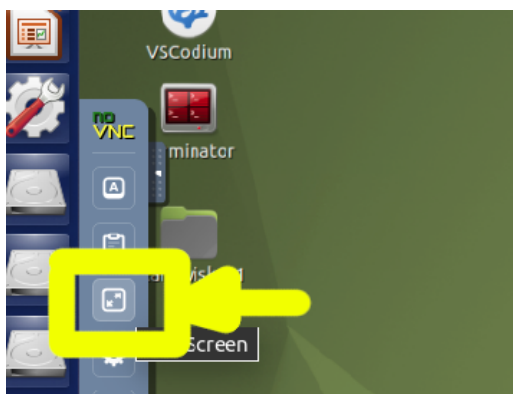
Aby otworzyć terminal w kontenerze, należy:

- kliknąć prawym przyciskiem myszy na pulpicie i wybrać „**Open in Terminal**”,
 - lub uruchomić program **Terminator** (ikona na pulpicie).
- Można przełączyć widok na pełny ekran, aby ukryć elementy systemu gospodarza i skupić się tylko na kontenerze.
- Z lewej strony na ekranie kontenera widoczny jest pasek boczny (lub mała strzałka, którą należy rozwinąć).



Rys. 2.9: Strzałka z lewej strony umożliwiająca rozwinięcie opcji dla kontenera.

- Po rozwinięciu paska należy wybrać trzecią opcję od góry – uruchamia ona środowisko pracy w trybie pełnoekranowym.



Rys. 2.10: Opcja pracy w trybie pełnoekranowym dla kontenera.

3 Konfiguracja sieciowa środowiska laboratoryjnego

Podczas pracy w laboratorium z kilkoma robotami jednocześnie niezwykle istotne jest prawidłowe skonfigurowanie parametrów komunikacyjnych ROS 2. Dzięki temu dane przesyłane pomiędzy robotem a komputerem PC nie będą się mieszały z komunikacją innych stanowisk.

3.1 ROS_DOMAIN_ID – separacja komunikacji

W ROS 2 komunikacja między węzłami odbywa się dzięki mechanizmowi **DDS (Data Distribution Service)**. Każdy zestaw urządzeń (robot + komputer PC) może działać w osobnej **domenie komunikacyjnej**, oznaczonej parametrem **ROS_DOMAIN_ID**.

- Jeśli dwa systemy mają różne **ROS_DOMAIN_ID**, ich komunikacja jest rozdzielona – nawet jeśli używają takich samych nazw tematów (topics).
- Dzięki temu w laboratorium możliwa jest równoczesna praca wielu robotów bez ryzyka kolizji komunikacji.
- Przykład: przy 5 robotach w tym samym laboratorium każda grupa pracuje w innej domenie, więc np. temat `/cmd_vel` jednego robota nie zakłóca działania pozostałych.
- **UWAGA! Jeśli robot zaczął się poruszać, a nie powinien, istnieje ryzyko, że inna grupa nieprawidłowo skonfigurowała ROS_DOMAIN_ID. W takiej sytuacji można zapytać inne grupy, czy nie próbują uruchomić programu i czy ich robot działa poprawnie. Jeśli nie, powinny one ponownie zweryfikować, czy stanowisko zostało uruchomione prawidłowo.**

3.2 Konfiguracja sieciowa robotów i stanowisk

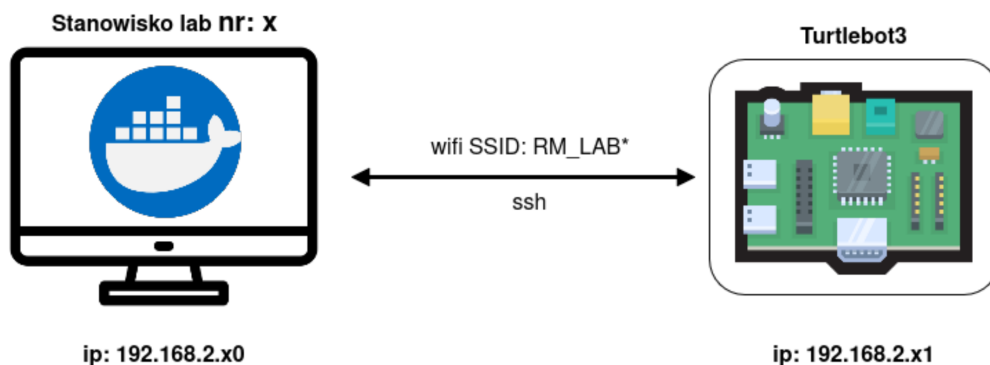
W laboratorium przygotowano pięć stanowisk, z których każde korzysta z innej platformy robota mobilnego:

- Stanowisko 1 – robot z napędem różnicowym *differential drive*,
- Stanowisko 2 – robot z napędem typu *skid-steer*,
- Stanowisko 3 – robot z napędem *car-like* (skręcane koła przednie),
- Stanowisko 4 – robot z napędem *omni (swerve drive)*,
- Stanowisko 5 – robot z napędem *omni (mecanum)*.

Dla każdego stanowiska przypisane są konkretne adresy **IP** oraz numery **ROS_DOMAIN_ID**. Grupa powinna znać:

- **IP** oraz **ROS_DOMAIN_ID** robota (Turtlebot3),
- **ROS_DOMAIN_ID** przypisany do stanowiska PC.

Na podstawie tabeli konfiguracyjnej Rys. 3.2 należy dobrać poprawne wartości.



Rys. 3.1: Schemat komunikacji z robotem.

Urządzenie	IP	ROS_DOMAIN_ID
Główny router Teltonika RUTM10	192.168.2.1	
PC stanowisko 1	192.168.2.10	10
PC stanowisko 2	192.168.2.20	20
PC stanowisko 3	192.168.2.30	30
PC stanowisko 4	192.168.2.40	40
PC stanowisko 5 	192.168.2.50	50
TB3_1 <u>RPI4</u> (2 wheel diff)	192.168.2.11	10
TB3_2 <u>RPI4</u> (skid-steer 4 wheels)	192.168.2.21	20
TB3_3 <u>RPI4</u> (car like 4 wheels)	192.168.2.31	30
TB3_4 <u>RPI4</u> (swerve drive 4 wheels)	192.168.2.41	40
TB3_5 <u>RPI4</u> (mecanum 4 wheels)	192.168.2.51	50
TB4_1 <u>RPI4</u> (Livox 360)	192.168.2.12	10
TB4_2 <u>RPI4</u> (OAK camera)	192.168.2.22	20
TB4_3 <u>RPI4</u>	192.168.2.32	30
TB4_4 <u>RPI4</u>	192.168.2.42	40
TB4_5 <u>RPI4</u>	192.168.2.52	50

Rys. 3.2: Konfiguracja sieci w laboratorium.

3.3 Konfiguracja pliku *.bashrc* w kontenerze

Plik */.bashrc* to skrypt wykonywany przy starcie każdej nowej sesji terminala w Bashu – służy do ustawiania zmiennych środowiskowych i aliasów. Dzięki wpisaniu tam np. `export ROS_DOMAIN_ID=30`, każdorazowe otwarcie terminala od razu ustawia właściwą domenę komunikacyjną ROS 2, co eliminuje potrzebę ręcznego konfigurowania przed uruchamianiem węzłów (nodów). Należy pamiętać, że jeśli w danym terminalu wykonujemy edycję tego pliku to, aby zmiany były widoczne należy uruchomić nowy terminal lub wpisać w aktualnym terminalu:

```
1 source ~/.bashrc
```

Kod źródłowy 5: Ponowne wczytanie pliku konfiguracyjnego */.bashrc*

Edycja pliku `.bashrc`

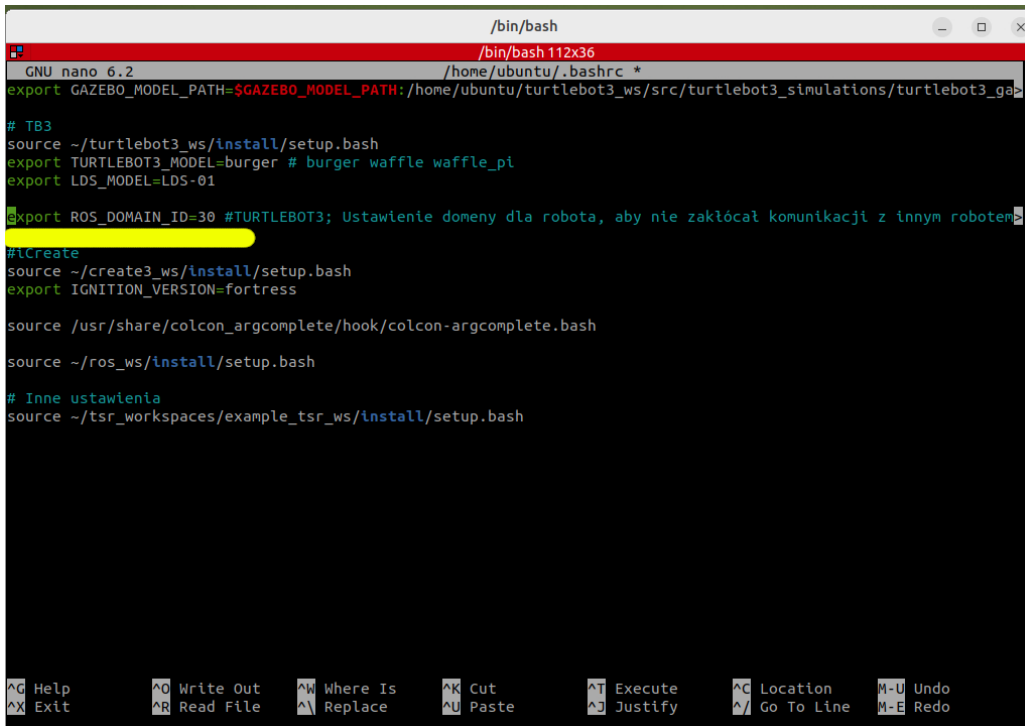
1. Otworzyć plik w edytorze **nano**:

```
1 nano ~/.bashrc
```

Kod źródłowy 6: Otwarcie pliku `/.bashrc` w edytorze nano

2. W pliku odnaleźć linię zawierającą **ROS_DOMAIN_ID**.

- Jeśli jest zakomentowana (`#export ROS_DOMAIN_ID=...`), należy usunąć znak `#`.
- Jeśli nie istnieje, należy dopisać ją ręcznie w dowolnym miejscu pliku. Najlepiej na końcu.



Rys. 3.3: Otwarty plik `/.bashrc` z zaznaczoną przykładową linią z ustawionym **ROS_DOMAIN_ID**.

3. Zwrócić uwagę na kolejność wpisów – jeśli w pliku zmienna **ROS_DOMAIN_ID** występuje kilka razy, ostatnia wartość jest nadrzędna.
4. Przykład: jeśli najpierw ustawiono **ROS_DOMAIN_ID=20**, a później **ROS_DOMAIN_ID=50**, to faktycznie aktywna będzie wartość **50**.
5. Zapisać zmiany i wyjść z edytora:

- **Ctrl+X**,
- następnie wpisać **y** (yes) i zatwierdzić **Enter**.

6. Aby zmiany były widoczne w aktualnym terminalu, należy wpisać:

```
1 source ~/.bashrc
```

Kod źródłowy 7: Ponowne wczytanie pliku konfiguracyjnego `/.bashrc`

lub otworzyć nowy terminal.

Po tej konfiguracji system będzie zawsze uruchamiał nowy terminal z poprawnym **ROS_DOMAIN_ID**, co gwarantuje poprawną komunikację z robotem przypisanym do stanowiska.

4 Uruchomienie środowiska Jupyter Notebook w Dockerze

Jupyter Notebook to interaktywne środowisko, które pozwala w jednym pliku pisać kod, wykonywać go krok po kroku oraz dokumentować wyniki. W ramach zajęć laboratoryjnych będziemy wykorzystywali język programowania **Python 3**.

4.1 Uruchamianie Jupyter Notebook

1. W terminalu w środowisku Dockera wpisz polecenie:

```
1 jupyter-notebook
```

Kod źródłowy 8: Polecenie służące do uruchomienia Jupyter Notebook.

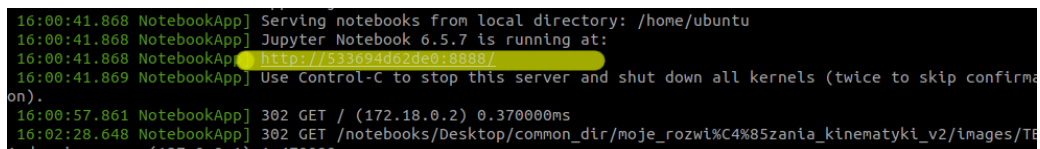
2. Środowisko uruchomi się w kontenerze na porcie **8888** i zostanie otwarte w przeglądarce.
3. Dla wygody można skopiować adres:

```
1 http://0.0.0.0:8888/tree
```

Kod źródłowy 9: Adres w przeglądarce pod, którym znajduje się Jupyter Notebook

4.2 Problemy przy uruchamianiu

- Jeżeli po wpisaniu polecenia *jupyter-notebook* środowisko nie otworzy się automatycznie, w terminalu zostanie wyświetlony adres, pod którym Jupyter Notebook działa.
- Adres ten znajduje się w linii zawierającej komunikat:



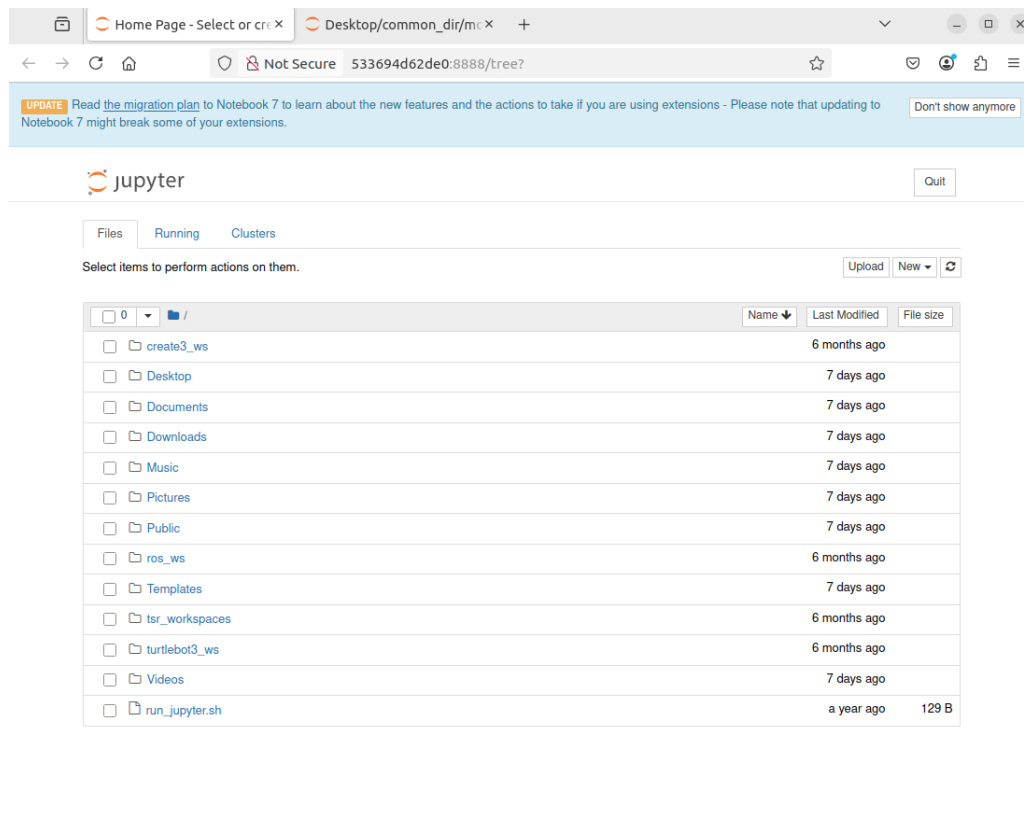
```
16:00:41.868 NotebookApp] Serving notebooks from local directory: /home/ubuntu
16:00:41.868 NotebookApp] Jupyter Notebook 6.5.7 is running at:
16:00:41.868 NotebookApp] http://533694dc2de0:8888/
16:00:41.869 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
on).
16:00:57.861 NotebookApp] 302 GET / (172.18.0.2) 0.370000ms
16:02:28.648 NotebookApp] 302 GET /notebooks/Desktop/common_dir/ moje_rozwi%C4%85zania_kinematyki_v2/images/Tf
drawings.png (172.18.0.2) 1.470000ms
```

Rys. 4.1: Adres w terminalu dla Jupyter Notebook.

- Wystarczy kliknąć prawym przyciskiem myszy na link i wybrać opcję **Otwórz w przeglądarce**.

4.3 Struktura katalogów w Jupyter Notebook

Po prawidłowym uruchomieniu notatnika widoczny będzie widok katalogów.



Rys. 4.2: Widok katalogów w Jupyter Notebook.

Należy przejść do:

```
1 Desktop/common_dir/studenckie
```

Kod źródłowy 10: Ścieżka do katalogu na prace studentów.

W tym miejscu powinien znajdować się wcześniej utworzony przez Państwa katalog.

4.3.1 Uruchamianie ćwiczeń

Ćwiczenia należy uruchamiać w kolejności – rozpoczynając od Ćwiczenia 1 przeznaczonego dla danego stanowiska.

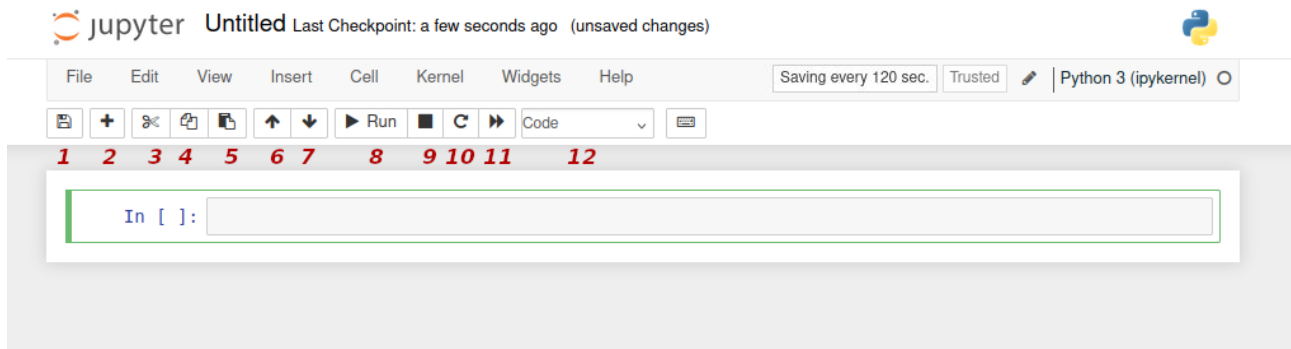
5 Obsługa Jupyter Notebook

W tym rozdziale przedstawiono najważniejsze elementy obsługi interfejsu **Jupyter Notebook** oraz typowe sytuacje, które mogą pojawić się podczas pracy.

5.1 Zapisywanie zmian

- Zmiany w notatniku zapisywane są **automatycznie**, ale można je też zapisać ręcznie.
- Jeśli w tytule pliku widnieje dopisek (**unsaved changes**), oznacza to, że nie wszystkie zmiany zostały zapisane.

5.2 Ikony i przyciski w pasku narzędzi



Rys. 5.1: Pasek narzędzi – Jupyter Notebook

1. **Zapisz** – przycisk do zapisu zmian
2. **+** – dodanie nowej komórki w notatniku.
3. **Nożyczki** – wycięcie zaznaczonej komórki (zostaje usunięta z widoku).
4. **Kopiuj** – skopiowanie zaznaczonej komórki.
5. **Wklej** – wklejenie wcześniej skopiowanej komórki poniżej aktualnie zaznaczonej.
6. **Strzałka w górę** – przesunięcie zaznaczonej komórki o jedną pozycję w górę.
7. **Strzałka w dół** – przesunięcie zaznaczonej komórki o jedną pozycję w dół.
8. **Run** – uruchomienie komórki.
 Jeśli komórka zawiera kod (np. w Pythonie), zostaje on wykonany, a wynik pojawia się poniżej.
 To najczęściej używana funkcja podczas zajęć.
9. **Interrupt the kernel** – przerywa aktualnie wykonywany kod, np. zatrzymuje nieskończoną pętlę.
10. **Restart the kernel** – uruchamia ponownie środowisko obliczeniowe (czyści pamięć i zmienne).
11. **Restart the kernel, then re-run the whole notebook** – restartuje środowisko i wykonuje wszystkie komórki od początku.
 Przydatne przy ponownym uruchamianiu całego pliku. Należy jednak uważać na komórki z komendami dotyczącymi testów komunikacji z robotem – mogą wymagać zakomentowania, gdyż uruchamiają nieskończony proces oznaczony *
12. Wybór typu komórki.

5.3 Typy komórek

- **Code** – do wpisywania i wykonywania kodu (np. w Pythonie). Wynik pojawia się pod spodem.
- **Markdown** – do opisu, dokumentacji i formatowanego tekstu (nagłówki, listy, wzory w LaTeX).
- **Raw** – zawiera surowy tekst, zapisywany bez przetwarzania.
- **Heading** – dawniej używana do nagłówków, obecnie zastąpiona nagłówkami w Markdown.

W materiałach dostarczonych do zajęć większość komórek ma już ustawiony właściwy typ, jednak zawsze warto sprawdzić, czy kod uruchamiany jest w komórce typu **Code**.

5.4 Numeracja komórek

- Obok każdej komórki znajduje się pole **In []**.
- Po uruchomieniu pierwszej komórki pojawia się numer **In [1]**.
- Kolejne uruchomienia są numerowane: **In [2]**, **In [3]**, itd.
- Numer oznacza, że kod został wykonany i można przejść do następnej komórki.

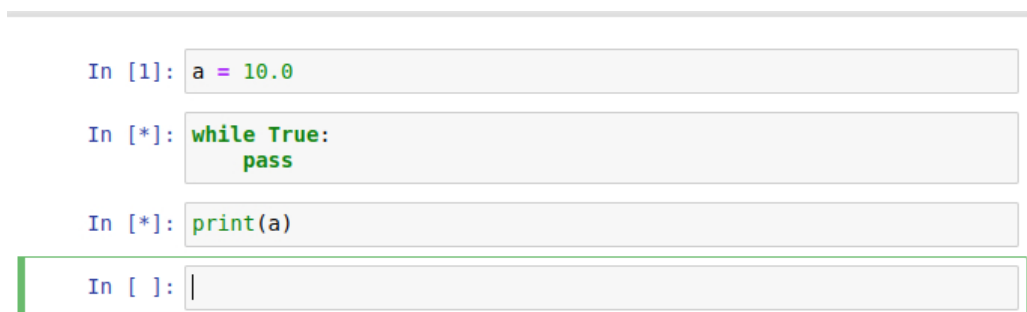


Rys. 5.2: Uruchomiona komórka **In [1]** w Jupyter Notebook.

5.5 Komórki z **In [*]** – ważne ostrzeżenie

Uwaga na występowanie **In [*]**, gdyż są one częstą przyczyną problemów studentów związaniem z niedziałaniem kodu, który uruchamiają. Jeśli program jest uruchomiony świadomie i powinien działać w pętli to nie jest to problem.

- Symbol **[*]** oznacza, że kod w tej komórce ciągle się wykonuje.
- Dopóki nie zakończy się działanie programu, nie można uruchomić innych komórek.
- Najczęstsza przyczyna to nieskończona pętla.



Rys. 5.3: Uruchomiona komórka **In [*]** z nieskończoną pętlą w Jupyter Notebook.

W tym przypadku program nigdy się nie kończy – dlatego przy **In []** pojawia się **[*]**. Próba uruchomienia innej komórki (np. wyświetlenia zmiennej **a**) nie powiedzie się, dopóki pętla nie zostanie przerwana.

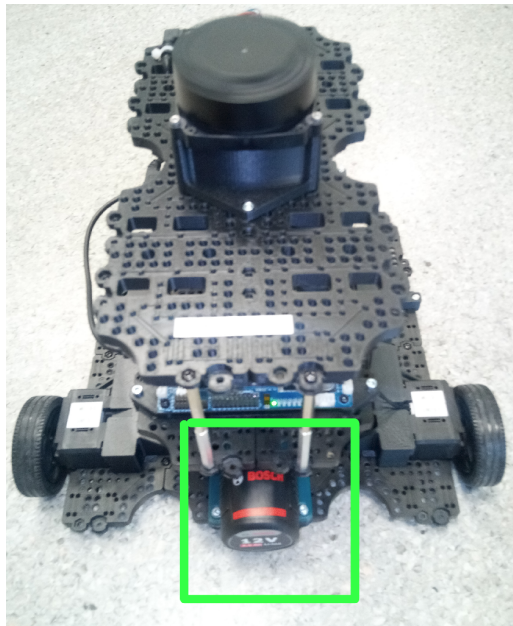
**Rozwiązanie: użyć przycisku Stop (Interrupt the kernel),
a następnie ponownie uruchomić kod.**

6 Uruchomienie robota

Przed rozpoczęciem pracy z robotem należy pamiętać, że jego czas działania jest ograniczony przez baterię. Dlatego zanim włączysz **robota**, **przygotuj kod i środowisko**, które będą potrzebne podczas ćwiczenia.

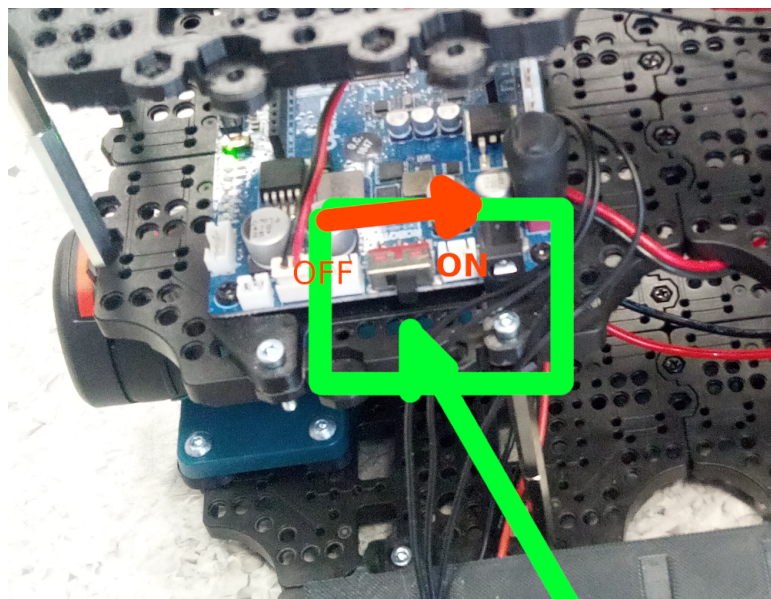
6.1 Przygotowanie robota

1. Umieścić naładowaną baterię w robocie.



Rys. 6.1: Umieszczenie baterii w robocie.

2. Przełącz przełącznik zasilania na niebieskiej płytce **OpenCR** w pozycję **ON** tzn. w prawo patrząc z przodu na przełącznik.



Rys. 6.2: Przełącznik na OenCR

3. Robot rozpocznie proces uruchamiania – należy odczekać około **1 minuty**, aż pojawi się w sieci.

6.2 Połączenie z robotem

1. W środowisku Dockera otwórz terminal.
2. Połącz się z robotem, wpisując:

```
1 ssh pi@192.168.2.x1
```

Kod źródłowy 11: Przykładowe polecenia do połączenia z robotem

gdzie **x** oznacza numer stanowiska.

3. Hasło do logowania:

```
1 ubuntu
```

Kod źródłowy 12: Hasło do robota

4. Po poprawnym zalogowaniu terminal pokaże informacje o systemie.

```

pi@pi: ~
pi@pi: ~ 113x39
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-1063-raspi aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Mar 31 13:42:36 UTC 2025

System load:          0.08
Usage of /:            26.1% of 28.72GB
Memory usage:         5%
Swap usage:           0%
Temperature:          45.3 C
Processes:            138
Users logged in:      0
IPv4 address for wlan0: 192.168.2.11
IPv4 address for wlan0: 192.168.2.117
IPv6 address for wlan0: fd9f:f6f2:48e::727
IPv6 address for wlan0: fd9f:f6f2:48e:0:2ecf:67ff:fe5b:dfc1

Expanded Security Maintenance for Applications is not enabled.

11 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

29 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Mon Mar 31 12:31:17 2025 from 192.168.2.113
pi@pi:~$

```

Rys. 6.3: Widok terminala po zalogowaniu się na robota.

Uwaga: łatwo odróżnić terminal Dockera od terminala robota:

- na robocie widać **pi@pi: \$**,
- w Dockerze widać **ubuntu@ciąg_cyfr:/ \$**.

6.3 Konfiguracja środowiska na robocie

Tak samo jak w Dockerze, należy skonfigurować plik **.bashrc**:

1. Otwórz plik:

```
1 nano ~/.bashrc
```

Kod źródłowy 13: Otwarcie pliku **/.bashrc** w edytorze nano.

2. W pliku odnaleźć linię zawierającą **ROS_DOMAIN_ID**.

- Jeśli jest zakomentowana (**#export ROS_DOMAIN_ID=...**), należy usunąć znak **#**.
- Jeśli nie istnieje, należy dopisać ją ręcznie w dowolnym miejscu pliku. Najlepiej na końcu.

```
1 export ROS_DOMAIN_ID=XX
```

Kod źródłowy 14: Linia w pliku `/.bashrc` z konfiguracją **ROS_DOMAIN_ID**.

gdzie **XX** to numer przypisany do stanowiska – musi być taki sam jak w środowisku Dockera w `/.bashrc`.

3. Zapisać zmiany i wyjść z edytora:

- **Ctrl+X**,
- następnie wpisać **y** (yes) i zatwierdzić **Enter**.

4. Aby zmiany były widoczne w aktualnym terminalu, należy wpisać:

```
1 source ~/.bashrc
```

Kod źródłowy 15: Ponowne wczytanie pliku konfiguracyjnego `/.bashrc`

5. Aby uruchomić niezbędne węzły ROS 2 odpowiedzialne m.in. za czujniki i sterowanie napędami, wpisz w terminalu:

```
1 ros2 launch turtlebot3_bringup RM_lab.launch.py
```

Kod źródłowy 16: Launch służący do uruchomienia oprogramowania na robocie.

- Po tej komendzie robot zostaje uruchomiony.
- W tym terminalu **nie wpisujemy już żadnych innych poleceń** – będzie on wyświetlał logi działania robota.

Robot jest gotowy do pracy.

7 Sterowanie robotem

Sterowanie robotem może odbywać się na różne sposoby – zarówno za pomocą interfejsów graficznych, jak i z poziomu terminala. W ramach zajęć najczęściej wykorzystywany będzie **graficzny interfejs przygotowany w Jupyter Notebook**, ponieważ jest on najbardziej wygodny i pozwala w prosty sposób zadawać prędkości.

Szczegóły dotyczące wysyłanych wiadomości znajdują się w oddzielnej instrukcji 3 opisującej architekturę komunikacyjną pomiędzy robotem i innymi węzłami ROS uruchamianymi w trakcie zajęć.

Niezależnie od wybranej jednej z poniższych metod, sterowanie zawsze odbywa się poprzez wysyłanie wiadomości typu *geometry_msgs/Twist* na temat `/cmd_vel`.

7.1 Interfejs graficzny w Jupyter Notebook

Należy uruchomić przygotowany interfejs sterowania znajdujący się w pliku `robot_control_interface.ipynb`.

Linear X:

Linear Y:

Angular Z:

Topic freq: Hz

Step sizes: Linear Step: Angular Step:

Current Velocity:
Linear: x=0.0, y=0.0
Angular: z=0.0

```
[INFO] [1742318738.518295167] [velocity_publisher]: Velocity reset to zero.
[INFO] [1742318752.154663284] [velocity_publisher]: Velocity reset to zero.
[INFO] [1742318764.571995319] [velocity_publisher]: Velocity reset to zero.
```

Rys. 7.1: Sterowanie z jupyter notebook.

Interfejs dostępny w Jupyter Notebooku pozwala w intuicyjny sposób zadawać wartości prędkości liniowej i kątowej robota, bez konieczności wpisywania komend w terminalu.

7.2 Sterowanie z terminala

Robotem można sterować również ręcznie poprzez terminal. Przykład wysłania wiadomości na temat `/cmd_vel` z użyciem komendy `ros2 topic pub`:

```
1 ros2 topic pub /cmd_vel geometry_msgs/msg/Twist
2 linear:
3   x: 0.5
4   y: 0.0
5   z: 0.0
6 angular:
7   x: 0.0
8   y: 0.0
9   z: 0.2
```

Kod źródłowy 17: Wysłanie przykładowych prędkości sterujących do robota z poziomu terminala.

Ta komenda ustawia prędkość liniową 0.5 m/s oraz prędkość obrotową 0.2 rad/s.

```
ubuntu@lukasz-ThinkPad-T480:~$ ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.2}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
^Cubuntu@lukasz-ThinkPad-T480:~$
```

Rys. 7.2: Sterowanie z terminala.

7.3 Interfejs graficzny *rqt_publisher*

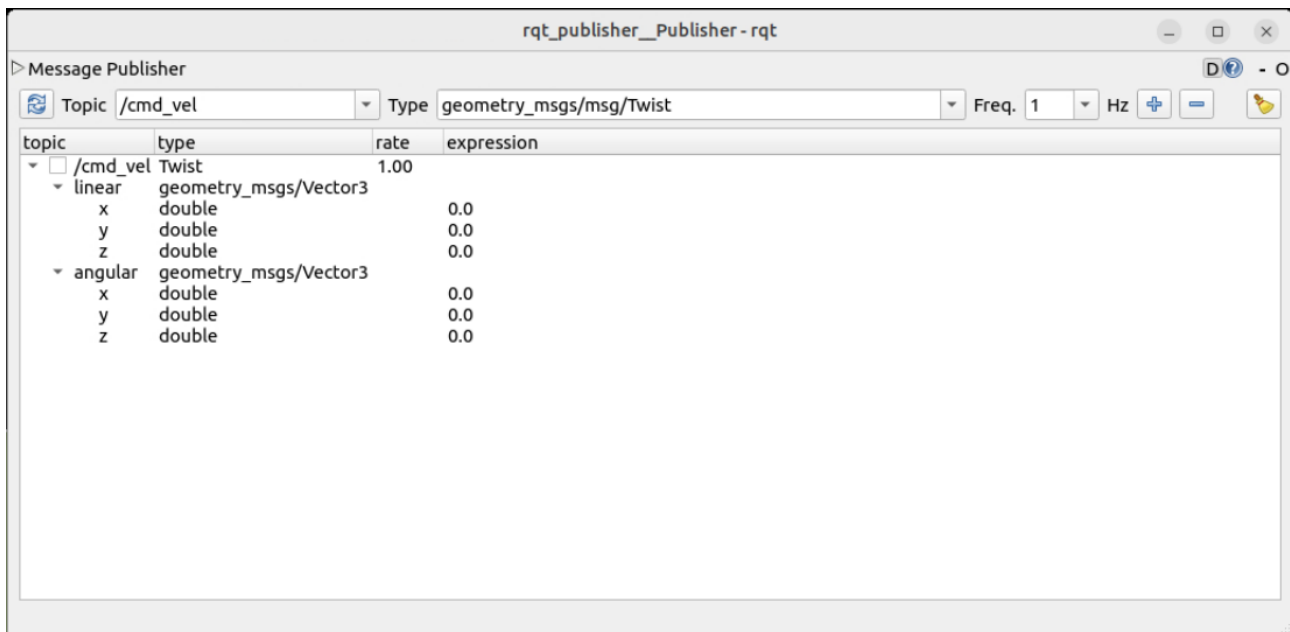
Innym sposobem jest wykorzystanie standardowego narzędzia ROS 2 – *rqt_publisher*.

Aby uruchomić program, wpisz w terminalu w kontenerze:

```
1 ros2 run rqt_publisher rqt_publisher
```

Kod źródłowy 18: Komenda w terminalu do uruchomienia *rqt_publisher*

Program otwiera graficzne okno, w którym można ręcznie wprowadzać wartości do wiadomości i wysyłać je do wybranego tematu. Z listy rozwijanej **Topic** należy wybrać `/cmd_vel`



Rys. 7.3: Sterowanie z rqt_publisher.

8 Weryfikacja działania kinematyki w ROS 2

Podczas pracy w środowisku ROS 2 można w prosty sposób sprawdzić, czy uruchomione oprogramowanie działa poprawnie i wymienia dane. Weryfikacja **nie sprawdza poprawności obliczeń**, ale pozwala upewnić się, że węzły zostały uruchomione i publikują odpowiednie komunikaty.

Wszystkie polecenia należy wpisywać w terminalu w **Dockerze**.

8.1 Weryfikacja kinematyki odwrotnej

1. Uruchom kod kinematyki odwrotnej. (Plik z wykonanym ćwiczeniem nr 2.)
2. Wyślij prędkości sterujące robotem na temat `/cmd_vel` np. poprzez interfejs graficzny Jupyter Notebook.
3. Otwórz nowy terminal i wpisz (krok opcjonalny):

```
1 ros2 topic echo /cmd_vel
```

Kod źródłowy 19: Nasłuchiwanie wiadomości ROS pojawiających się na temacie `/cmd_vel`.

- To polecenie pozwala sprawdzić, czy interfejs użytkownika wysyła poprawnie prędkości sterujące robotem.
- Brak publikacji na tym temacie uniemożliwia obliczenie prędkości dla poszczególnych kół.

4. W kolejnym terminalu sprawdź, czy węzeł kinematyki odwrotnej publikuje przeliczone prędkości kół:

```
1 ros2 topic echo /cmd_joint_states
```

Kod źródłowy 20: Nasłuchiwanie wiadomości ROS pojawiających się na temacie `/cmd_joint_states`.

- Jeśli wszystko działa poprawnie, pojawiają się wartości prędkości dla poszczególnych kół.
- Można zmienić prędkości sterujące w interfejsie i zaobserwować, czy zmieniają się wartości publikowane na temat `/cmd_joint_states`.

5. Jeśli wszystko działa poprawnie można uruchomić rozwiązanie na rzeczywistym robocie.

8.2 Weryfikacja kinematyki prostej

- Uruchom kod kinematyki prostej. (Plik z wykonanym ćwiczeniem nr 3.)
- Sprawdź w terminalu, czy odbierane są dane z czujników kół (m.in. ich prędkości i pozycje):

```
1 ros2 topic echo /joint_states
```

Kod źródłowy 21: Nasłuchiwanie wiadomości ROS pojawiających się na temacie `/joint_states`.

- Brak danych oznacza, że odometria nie zostanie obliczona.

- Sprawdź w terminalu, czy węzeł kinematyki prostej publikuje dane dotyczące odometrii:

```
1 ros2 topic echo /odometry
```

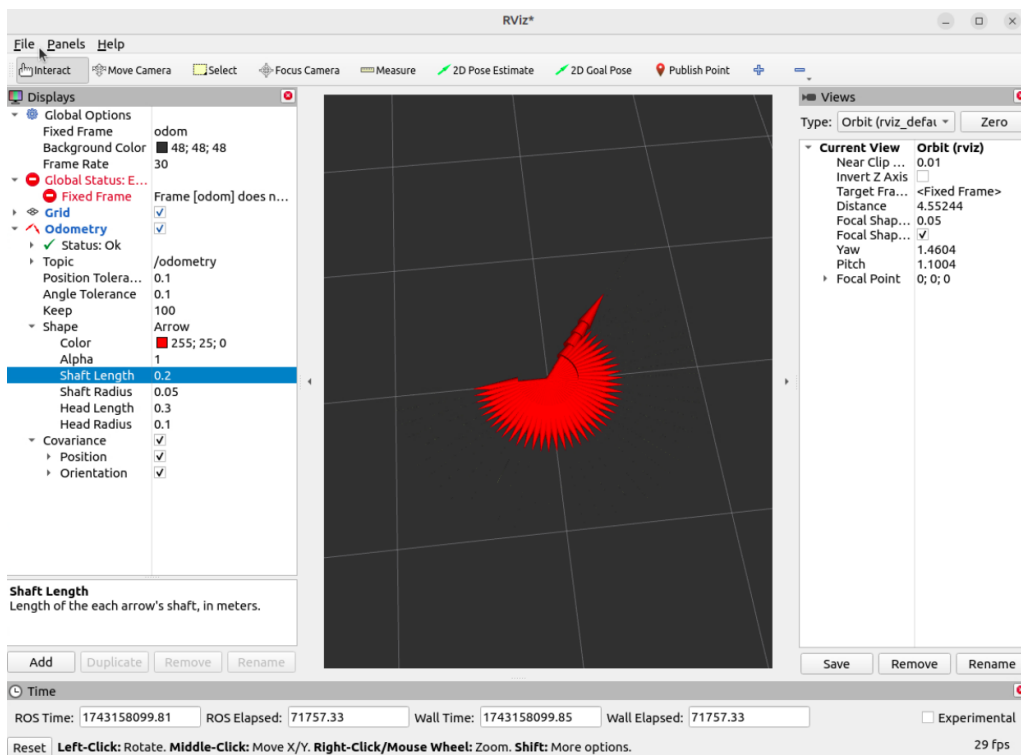
Kod źródłowy 22: Nasłuchiwanie wiadomości ROS pojawiających się na temacie `/odometry`.

- Jeżeli dane są widoczne, oznacza to, że węzeł działa poprawnie i oblicza pozycję robota na podstawie informacji z kół.

- Zweryfikuj poprawność działania odometrii w sposób graficzny, uruchamiając program **RViz2**:

```
1 rviz2
```

Kod źródłowy 23: Uruchomienie programu **RViz2** do wizualizacji danych



Rys. 8.1: Wizualizacja danych za pomocą **RViz2**

- W polu **Global Options** w **Fixed Frame** ustaw ramkę *odom*, a jeśli nie istnieje to należy ją wpisać ręcznie.
- W oknie programu kliknij **Add** i wybierz temat */odometry*.
- Program wyświetli trajektorię ruchu robota, co pozwala ocenić poprawność wyliczania odometrii.
- **RViz2** stanowi wygodne uzupełnienie weryfikacji danych z terminala, gdyż umożliwia ich wizualizację.